Prueba de Concepto (PoC): Despliegue Automatizado de un Señuelo con Terraform y Docker.

Proyecto: Honeynet Adaptativo con Verificación Automática de Superficie de Ataque.

Alumno: Adrián Mirabent Landero.

Centro: IES La Marisma (Huelva).

Sumario

1. Introducción y Entorno de Trabajo....3
2. Definición de la Infraestructura como código....4
3. Análisis de los Componentes del Despliegue....5
4. Grafo de Dependencias y Orquestación....7
5. Ejecución y Despliegue Automatizado....7
6. Destrucción Limpia del Señuelo....8
7. Reconstrucción y Verificación....9

1. Introducción y Entorno de Trabajo.

Para validar de forma empírica el funcionamiento del proyecto, se ha desarrollado una Prueba de Concepto (PoC) inicial sobre una máquina virtual en ProxMox con el sistema operativo Ubuntu 22.04 LTS.

El objetivo de esta prueba es demostrar como aprovisionar mediante código una infraestructura de señuelos aislada de forma automatizada.

En primer lugar, se procede a la instalación de los componentes base de automatización e infraestructura:

- Terraform: Herramienta de Infraestructura como Código (IaaC) para la declaración y gestión del ciclo de vida de los recursos.
- Docker: Motor de la contenerización encargado de ejecutar los servicios de forma aislada.

Los comandos de aprovisionamiento e inicialización del entorno ejecutados en la terminal son los siguientes:

1) Descarga e instalación de la clave y repositorio oficial de HashiCorp (Terraform).

```
adrian@adrian-Standard-PC-i440FX-PIIX-1996:~$ wget -O- https://apt.releases.hashicorp.com/gpg |
 sudo gpg --dearmor -o /usr/share/keyrings/hashicorp.gpg
```

```
adrian@adrian-Standard-PC-i440FX-PIIX-1996:~$ echo "deb [signed-by=/usr/share/keyrings/hashicorp.gpg]
 https://apt.releases.hashicorp.com $(lsb_release -cs) main" | sudo tee /etc/apt/sources.list.d/hashi
corp.list
```

2) Actualización e instalación de Terraform y Docker.

```
adrian@adrian-Standard-PC-i440FX-PIIX-1996:~$ sudo apt update && sudo apt install -y terraform docker.io
```

3) Configuración de permisos de usuario para la gestión de contenedores. Y reinicio.

```
adrian@adrian-Standard-PC-i440FX-PIIX-1996:~$ sudo usermod -aG docker $USER
```

Para verificar que el entorno está operativo, se comprueba la versión de Terraform y el estado del demonio de Docker, los cuales deben responder sin errores de permisos.

```
adrian@adrian-Standard-PC-i440FX-PIIX-1996:~$ terraform -version
Terraform v1.16.6
on linux_amd64
```

```
adrian@adrian-Standard-PC-i440FX-PIIX-1996:~/senuelo1$ docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
```

2. Definición de la Infraestructura como código.

Se crea un directorio de trabajo específico (/home/adrian/senuelo1) donde se ubicará el fichero de configuración declarativo main.tf. Este fichero automatiza por completo la descarga de dependencias, la creación de una red virtual privada y el despliegue del primer contenedor señuelo con un servidor web ligero (Nginx Alpine).

El código fuente del archivo main.tf es el siguiente:

adrian@adrian-Standard-PC-i440FX-PIIX-1996: ~/senuelo1

GNU nano 6.2 main.tf

```
# Bloque de configuración general de Terraform
terraform {
        required_providers {
                docker = {
                        source = "kreuzwerker/docker"
                        version = "~> 3.0"
                }
        }
}

# Bloque del proveedor
provider "docker" {}

# Bloque de recurso. Creación de una imagen de Docker
resource "docker_image" "nginx" {
        name = "nginx:alpine"
}

# Bloque de recurso. Creación de una red de Docker
resource "docker_network" "honeynet" {
        name = "honeynet"
        driver = "bridge"
        ipam_config {
                subnet = "172.30.0.0/24"
        }
}

# Bloque de recurso. Creación del contenedor de Docker
resource "docker_container" "web_senuelo" {
        name = "web-senuelo"
        image = docker_image.nginx.image_id
        networks_advanced {
                name = docker_network.honeynet.name
        }
}
```

3. Análisis de los Componentes del Despliegue.

Voy a explicar el contenido de cada bloque.

Bloque de configuración general de Terraform.

Dentro de required_providers, se declara los proveedores externos que Terraform necesitará descargar y utilizar.

Dentro del proveedor de Docker se declara la configuración.

En source se define el origen del proveedor de docker en el registro de Terraform.

En version queda registrado la versión requerida.

```
# Bloque de configuración general de Terraform
terraform {
        required_providers {
                docker = {
                        source = "kreuzwerker/docker"
                        version = "~> 3.0"
                }
        }
}
```

Bloque del proveedor.

Inicializa y configura el proveedor de Docker. Al dejar las llaves vacías, le indica a Terraform que utilice los valores por defecto.

```
# Bloque del proveedor
provider "docker" {}
```

Bloque de recurso. Creación de una imagen de Docker.

Declara la creación de un recurso tipo docker_image, con identificador local nginx.

En name se especifica el parámetro principal de la imagen a descargar:

utiliza la imagen oficial de Nginx con la etiqueta alpine (versión más ligera).

```
# Bloque de recurso. Creación de una imagen de Docker
resource "docker_image" "nginx" {
        name = "nginx:alpine"
}
```

Nginx es un servidor web de código abierto, ligero y de alto rendimiento.

Bloque de recurso. Creación de una red de Docker.

Declara la creación de un recurso tipo docker_network, con identificador local honeynet.

En name se declara el nombre real de la red en Docker.

En driver se declara el tipo de controlador de red tipo bridge (el estándar que se usa para aislar varios contenedores en un mismo host).

En ipam_config se recoge la configuración de direcciones de la red.

Con subnet se le asigna un rango de subred específico para los dispositivos conectados a esa red.

```
# Bloque de recurso. Creación de una red de Docker
resource "docker_network" "honeynet" {
        name = "honeynet"
        driver = "bridge"
        ipam_config {
                subnet = "172.30.0.0/24"
        }
}
```

Bloque de recurso. Creación del contenedor de Docker.

Declara la creación de un recurso tipo docker_container, con identificador local web_senuelo.

En name se define el nombre del contenedor.

En image asocia el contenedor a la imagen de Nginx definida en un bloque de recurso anterior, utilizando su identificador único (image_id).

En networks_advanced se configura la red para el contenedor.

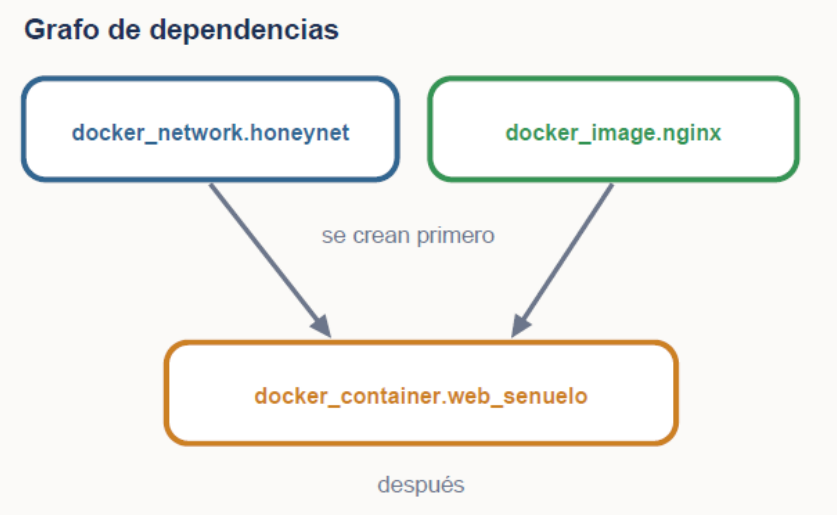
En name conecta este contenedor a la red definida en un bloque de recurso anterior.

```
# Bloque de recurso. Creación del contenedor de Docker
resource "docker_container" "web_senuelo" {
        name = "web-senuelo"
        image = docker_image.nginx.image_id
        networks_advanced {
                name = docker_network.honeynet.name
        }
}
```

4. Grafo de Dependencias y Orquestación.

Gracias al motor de análisis de Terraform, el sistema comprende de manera automática el orden de preferencia necesario para levantar la infraestructura sin errores:

- Fase 1: Terraform evalúa y ejecuta primero la creación de la red virtual (docker_network.honeynet) y la descarga de la imagen base (docker_image.nginx).
- Fase 2: Una vez validados y creados los recursos previos, se ejecuta el despliegue del contenedor (docker_container.web_senuelo) utilizando los identificadores generados en la fase anterior.



5. Ejecución y Despliegue Automatizado.

Una vez redactado el fichero de configuración, el ciclo de vida del despliegue se ejecuta mediante la secuencia estándar de comandos de Terraform:

- terraform init: Inicializa el directorio y descarga el proveedor de Docker.
- terraform plan: Muestra una vista previa de los recursos que se van a crear.
- terraform apply: Ejecuta la creación real de la infraestructura.

```
adrian@adrian-Standard-PC-i440FX-PIIX-1996:~/senuelo1$ terraform init
adrian@adrian-Standard-PC-i440FX-PIIX-1996:~/senuelo1$ terraform plan
adrian@adrian-Standard-PC-i440FX-PIIX-1996:~/senuelo1$ terraform apply
```

Una vez realizado todos estos pasos, el contenedor queda levantado, conectado a su subred y listo.

Con el comando terraform state list, podemos ver todos los bloques que está gestionando Terraform.

```
adrian@adrian-Standard-PC-i440FX-PIIX-1996:~/senuelo1$ terraform state list
docker_container.web_senuelo
docker_image.nginx
docker_network.honeynet
```

Con el comando terraform show podemos ver en detalle el contenido de los recursos activos en el sistema.

```
adrian@adrian-Standard-PC-i440FX-PIIX-1996:~/senuelo1$ terraform show
```

6. Destrucción Limpia del Señuelo.

Dado que el objetivo principal de un honeypot o señuelo es interactuar con agentes maliciosos y correr el riesgo de ser comprometido, una de las ventajas clave de este proyecto es la resiliencia operativa.

Para eliminar de manera radical y limpia cualquier rastro de alteración, compromiso o código malicioso que el atacante haya podido inyectar en el contenedor, se ejecuta el comando de destrucción automatizada:

```
adrian@adrian-Standard-PC-i440FX-PIIX-1996:~/senuelo1$ terraform destroy
```

Vamos a ver los bloques que está gestionando ahora Terraform.

```
adrian@adrian-Standard-PC-i440FX-PIIX-1996:~/senuelo1$ terraform state list
adrian@adrian-Standard-PC-i440FX-PIIX-1996:~/senuelo1$
```

Vemos que no está gestionando nada.

7. Reconstrucción y Verificación.

Una vez destruida la infraestructura, se puede restaurar el entorno por completo a su estado inicial, limpio y seguro, mediante un único comando:

```
adrian@adrian-Standard-PC-i440FX-PIIX-1996:~/senuelo1$ terraform apply
```

Vamos a ver los bloques que está gestionando ahora Terraform. Tenemos creada de nuevo toda la infraestructura.

```
adrian@adrian-Standard-PC-i440FX-PIIX-1996:~/senuelo1$ terraform state list
docker_container.web_senuelo
docker_image.nginx
docker_network.honeynet
```

Esto demuestra empíricamente la recuperación ante desastres (Disaster Recovery). Cualquier señuelo vulnerado puede ser destruido y reemplazado por una réplica limpia, idéntica y funcional de forma automatizada y casi instantánea.